GEDCOM Examples

Leading White Space in Examples

GEDCOM does not allow GEDCOM lines to have any leading white space. The GEDCOM specification should lead by example; all examples within the specification should be valid examples. No more “do as we say, not as we do”. All leading white has been removed from the examples.

Best Practices

Some GEDCOM examples contained bad practice such as incomplete or abbreviated place names. The GEDCOM specification should promote best practices. The examples have been improved to use complete, unabbreviated place names.

Obsolete and Superfluous Records

Ancestral File

Ancestral File is a long retired system. All records in support of Ancestral File are obsolete, and have been removed. This includes the <SUBMISSION_RECORD> and the SUBM.RFN record.

SUBM.LANG

The SUBM.LANG record, which allowed specifying a language preferred by the submitter, does not serve a clear purpose. The GEDCOM header already provides HEAD.LANG to specify the language used in the GEDCOM file, and specifying a preferred language only makes sense if you expect a response. The SUBM.LANG field is superfluous at best, and not used in practice. The *GEDCOM 5.5.1 Annotated Edition* marks it as deprecated, GEDCOM 5.5.5 removes its.

<PLACE_HIERARCHY>

The intended complexity of the <PLACE_HIERARCHY> feature was insufficiently defined and remained widely unsupported. The <PLACE_HIERARCHY> feature has been removed. All place names must be specified as comma-and-space separated place parts, listed from smallest to largest jurisdiction.

<SOURCE_CITATION>

GEDCOM 5.5.1 already deprecated, but still allowed the old <SOURCE_CITATION> format in addition to the new one introduced in GEDCOM 5.4 (1995 CE). GEDCOM 5.5.5 simplifies source citations by allowing just the “new” source citation format, which includes a reference to a SOUR record.

<SOURCE_REPOSITORY_CITATION>

The GEDCOM 5.5.1 <SOURCE_REPOSITORY_CITATION> structure definition is really two alternate structure definitions that are presented as one. An odd side-effect of this confusing presentation is that, upon first reading, the <SOURCE_REPOSITORY_CITATION> structure definition seems to be the only record in the GEDCOM 5.5.1 specification that expect a pointer value, yet allows it to be absent (<NULL>). That is not the case. What is really going is that GEDCOM 5.5.1 supports source citations that do reference repositories and source citations that do not. The *GEDCOM 5.5.1 Annotated Edition* shows the split into the two different definitions. GEDCOM 5.5.5 simplifies <SOURCE_REPOSITORY_CITATION> (which is an *optional* subrecord of <<SOURCE_RECORD>>) by demanding that, if present, it references a repository, as its name suggest.

One Submitter per File

GEDCOM no longer allows individual records to labelled with submitters. This feature was not used in practice, because it has no practical usage, not even for shared trees. GEDCOM 5.5.5 keeps it simple by allowing only one submitter, and therefore only one <SUBMITTER_RECORD>, for the entire file.
This makes the HEAD.SUBM reference to that single submitter record superfluous, but it is kept for compatibility's sake.

One Multimedia Record per File, one File per Multimedia Record

GEDCOM 5.5.1 allows multiple media files per multimedia record. This is unnecessary, and remains poorly supported, so export to GEDCOM import into another system is likely to result in data loss.
GEDCOM 5.5.5 solves that data loss issue by simplifying multimedia support. GEDCOM 5.5.5 demands one multimedia record per file, one file per multimedia record.

SSN

GEDCOM version 5.3 through 5.5.1 define a SSN record for the U.S. Social Security Number, duplicating the already existing IDNO record functionality.
GEDCOM 5.5.5 uses the IDNO for all third-party numbers. Application can identify Social Security numbers with the IDNO.TYPE value “SSN”.

It is best practice for GEDCOM writers using GEDCOM 5.5.1 or earlier to ignore the SSN record, and always use the IDNO record, just like a GEDCOM 5.5.5 writer does.

Fixes

EMAIL not EMAI

The EMAIL record always uses an EMAIL tag. The confusing EMAI (no L) definition in Appendix A has been replaced with an EMAIL definition.

single <CHANGE_DATE>

Replaced two different <CHANGE_DATE> definitions (GEDCOM 5.5.1 page 31 and 44) with a single <CHANGE_DATE> definition

ASSO Definition in ASSO Definition

Moved ASSO definition and example from <INDIVIDUAL_RECORD> definition to <ASSO_RECORD> definition.

Maximum Length Multimedia Files Names

Fixed the maximum length of multimedia files names, from 30 to 259 code units.

DATE Spaces

Dates contain spaces. Fixed all the date definitions to include spaces in the syntax.

Hebrew B.C.

Removed the erroneous “B.C.” from the Hebrew date syntax.

Dual-Style Dates: Julian Calendar

England and its colonies used dual-style dates with the Julian Calendar. However, FamilySearch GEDCOM 5.4 through GEDCOM 5.5.1 restricts dual-style dates to the Gregorian Calendar... GEDCOM 5.5.5 allows dual-style Julian Calendar dates.

@#DROMAN@

The escape sequence `@#DROMAN@` has been removed from <DATE_CALENDAR_ESCAPE> because there is no associated calendar.

<PERSONAL_NAME_PIECES> Mandatory

To prevent unnecessary loss of information on export & import, the NPFX, GIVN, NICK, SPFX, SURN, and NSFX subrecords that identify name parts of the <PERSONAL_NAME_STRUCTURE> are no longer optional. Each of the subrecords remains optional, because no name parts occurs in every recorded name, but all name parts must be clearly identified.

<MULTIMEDIA_FORMAT> Fixes

Removed `OLE` from the list of multimedia formats: OLE is not a multimedia format at all, it is a technology. GEDCOM 5.5 uses the values `JPEG` and `TIFF`, GEDCOM 5.5.1 uses `JPG` and `TIF` instead.
GEDCOM 5.5.5 uses the values `JPEG` and `TIFF`, not `JPG` and `TIF`. The `JPG` and `TIF` values are supported, but deprecated.
The list of allowed multimedia formats has been expanded, but the fact that the enumeration of allowed formats is supposed to be exhaustive remains a fundamental issue.

<AGE_AT_EVENT> Centenarian Support

The <AGE_AT_EVENT> definition allowed only 2 digits for the year. <AGE_AT_EVENT> allows three digits for the year now.

TYPE subrecords in <<INDIVIDUAL_ATTRIBUTE_STRUCTURE>>

The GEDCOM 5.5.1 <<INDIVIDUAL_ATTRIBUTE_STRUCTURE>> description states that the FACT and IDNO records *require* a TYPE subrecords, while it *may* be used with all the other record types, while there is no TYPE record is not in the syntax! GEDCOM 5.5.5 adds the TYPE subrecords to the syntax.

<INDIVIDUAL_EVENT_STRUCTURE>

The updated <INDIVIDUAL_EVENT_STRUCTURE> syntax no longer allows `1 BIRT Y`; we already know that individuals were born; the only thing that record offers is a chance for a GEDCOM reader to get confused by it. The BIRT record should only be used to provide real information.

Clarifications

GEDCOM Form is Case-Sensitive

Within GEDCOM, so-called controlled line values (exhaustive enumerations) are case-insensitive, but the GEDCOM form *is not* a controlled line value, quite the opposite. The GEDCOM form value is case-sensitive. The line value used for the Lineage-Linked form, `LINEAGE-LINKED`, must be written all-uppercase.

Submitter after Header

The syntax of Lineage-Linked Form has been clarified to explicitly demand that the one mandatory <SUBMITTER_RECORD> directly follow the <HEADER_RECORD>.

Leading Zeroes on Day

Resolved open issue: <DAY>, the day of the month within a date, may *not* start with leading zeroes.

Proper Version Numbers

GEDCOM versions up through GEDCOM 5.5.1 (and the GEDCOM 5.6 draft) defined <VERSION_NUMBER> as an arbitrary string. GEDCOM 5.5.5 demands that the HEAD.SOUR.VERS and HEAD.GEDC.VERS line values be version numbers, and nothing else. The version number records can no longer be abused to contain the product name, system id, release status or platform. GEDCOM 5.5.5 readers will encounter nothing but a version number.

Two Version Number Formats

GEDCOM 5.5.1 replaces the single <VERSION_NUMBER> definition with two separate definitions; <GEDCOM_VERSION_NUMBER> and <PRODUCT_VERSION_NUMBER>. <GEDCOM_VERSION_NUMBER> demands a version number in the *major.minor.revision*format. <PRODUCT_VERSION_NUMBER> demands a version number in the *major.minor.revision.build* format.

Default <SEX_VALUE>

The default <SEX_VALUE> is `U` for *Unknown*.

<PEDIGREE_LINKAGE_TYPE>

Kept the *GEDCOM 5.5.1 Annotated Edition* remark that GEDCOM does not support DNA or biological relationships.
Added statement that, in absence of <PEDIGREE_LINKAGE_TYPE>, the value `birth` for official parents is assumed.

New Features

HEAD.GEDC.FORM.VERS

The version number for the GEDCOM form, separate from the GEDCOM version number.

New SEX Values

- Added `X`, meaning *Intersex* as a possible <SEX_VALUE>
- Added `N`, meaning *Not recorded* as a possible <SEX_VALUE>

Birthdays: Dates without Years

The <DATE_GREG> definition has been expanded to include exact dates without a year; this acknowledges that we often know a birt hday while being unsure of the birth year.

BCE

The *GEDCOM 5.5.1 Annotated Edition* already recommended that GEDCOM readers accept both `BC` and `BCE` in addition to `B.C`. GEDCOM 5.5.5 makes the `BCE` and `BC` values official. All three values mean the same thing. The religiously neutral `BCE` is preferred, the other two values are deprecated. However, it is recommended that developers who want to maximise compatibility of their GEDCOM 5.5.1 and GEDCOM 5.5.5 output, use `B.C.` for now.

GEDCOM Writer & Reader Checklists

GEDCOM 5.5.5 enables simple, straightforward and much faster GEDCOM readers by demanding strict compliance from GEDCOM writers. GEDCOM 5.5.5 writers *must* produce correct GEDCOM, so that developers need not spend time complicating their GEDCOM 5.5.5 readers with support for all kinds of third-party errors. In fact, to really ease the burden, a GEDCOM 5.5.5 reader *must not* support erroneous import. This keeps GEDCOM 5.5.5 readers simple, and forces all developers to produce quality GEDCOM files. Developers that want to continue to produce invalid GEDCOM files, must keep using GEDCOM 5.5.1.

The following two checklists are offered as an aid to development.

GEDCOM 5.5.5 Writer

- *must* use the Unicode character set, using either the UTF-8 or UTF-16 encoding
- *must* make sure that it's UTF-8 output is really UTF-8, not CESU-8
- *must* start each GEDCOM file with Byte Order Mark (BOM)
- *must* use a proper system identifier
- *must* use a proper version number for its system
- *must* default HEAD.DEST to the same system identifier as HEAD.SOUR
- *must* specify GEDCOM version 5.5.5
- *need not* create a HEAD.SUBM record, but if it does, the pointer must be valid
- *should* specify GEDCOM form `LINEAGE-LINKED`, there is no other GEDCOM form
- *does not* output superfluous white space
- does not include either CR or LF inside line values, uses CONT records to communicate a line feed
- respects the maximum line length of 255 code units, uses CONC record to support long line values
- *does not* use CONC or CONT records in the GEDCOM header
- *never uses* CONC records for line values with a maximum length of 248 or less
- writes the submitter record directly after the GEDCOM header
- uses legal GEDCOM 5.5.5 records exclusively
- does not create any empty lines or leading white space
- Makes sure that every line value is a valid string in the encoding used, by not merely splitting lines between code units or code, but between characters

GEDCOM 5.5.5 Reader

- *must* demand that GEDCOM file starts with a Byte Order Mark (BOM), *must reject* the file as not GEDCOM 5.5.5, with a message such as “ostensible GEDCOM 5.5.5 file lacks Byte Order Mark”
- *must* demand a legal character encoding, *must not* tolerate illegal character sets or encodings
- *must* accept UTF-8, but *should* detect and reject CESU-8
- *must* check that the HEAD.CHAR value matches the encoding used, and abort with a fatal error when it does not
- *must* check that each line value is a valid string in the encoding use, and *must* reject the GEDCOM file if any string is invalid
- *must* reject a GEDCOM header without a GEDCOM form
- *must* check that the HEAD.SUBM pointer, if present, is valid
- *must not* try to process an GEDCOM form it does not explicitly support, but *must* abort with an “unsupported GEDCOM form” message
- *must not* accept lines longer than 255 code units, *must* reject the file as invalid, with an error message such as “Line is too long. This is not a GEDCOM 5.5.5 file.”.
- Accepts all line values *as-is*, *does not* trim leading or trailing white space from any line value
- processes CONC and CONT records correctly
- *must not* tolerate CONC or CONC records in the Basic GEDCOM header
- *must* accept CONC or CONC records in the GEDCOM form's header extension
- *must* demands that system identifiers are valid, *must not* tolerate known nonsense

values
- *must* demand legal records, *must not* tolerate illegal records
- *must* especially demand a valid GEDCOM header, *must not* accept any error in the header
- *must* import data as-is, *must not* modify anything during import
- *must not* tolerate empty lines or leading white space, but *must* reject such files as not GEDCOM.

Support for Earlier GEDCOM Versions

GEDCOM 5.5.5 succeeds GEDCOM 5.5.1 (1999 CE). As the GEDCOM 5.5.1 specification has been the *de facto* standard for two decades, there is no pressing need to support any older GEDCOM versions, not even GEDCOM 5.5.
Applications should take care to recognise GEDCOM 5.5.1 files with a truncated version number; GEDCOM 5.5.1 files that are incorrectly labelled as GEDCOM 5.5 files.

Application that already support GEDCOM 5.5.1, should add support for GEDCOM 5.5.5, while continuing to support GEDCOM 5.5.1. New applications should support GEDCOM 5.5.5 first, and developers should only add GEDCOM 5.5.1 support if there is still significant demand for it. GEDCOM export should default to the latest GEDCOM version.

Applications should maintain existing GEDCOM 5.5.1 export functionality. Applications should not offer export to GEDCOM versions older than GEDCOM 5.5.1.

Applications *must* detect the GEDCOM version of GEDCOM files before trying to read them, and *must not* try to read GEDCOM versions they do not support, but *must* honestly report unsupported versions as unsupported.

Applications should read older GEDCOM versions for compatibility's sake. Every application should be able to read the GEDCOM files it has written, including all GEDCOM files produced by earlier versions of that application, just as it should be able to read old versions of its own file format.

Developers who decide to drop import of GEDCOM versions before GEDCOM 5.5.1, should provide an alternative means to get these imported, for example by providing a free download of an older version that can import the older files, and then export to GEDCOM 5.5.1.

Introduction

GEDCOM is an acronym for **GE**nealogical **D**ata **Com**munication.
That name is a misnomer. GEDCOM neither is nor contains a communication protocol.
GEDCOM is a data file format.

GEDCOM was created by FamilySearch, a subsidiary of the The Church of Jesus Christ of Latter-day Saints (LDS), that was originally known as the Family History Department.

GEDCOM aims to provide a flexible, uniform format for exchanging computerised genealogical data.
Its purpose is to foster the sharing of genealogical information and the development of a wide range of interoperable software products to assist genealogists, historians, and other researchers.

Purpose and Content of *The GEDCOM Standard*

The GEDCOM Standard is a technical document written for computer programmers, system developers, and technically sophisticated users. It covers the following topics:

- Basic GEDCOM Language (see Chapter 1, page 46)
- Lineage-Linked Form (see Chapter 2, page 53)
- Character Sets in GEDCOM (see Chapter 3, page 112)
- Lineage-Linked GEDCOM Tags (see Appendix A, page 121 and Chapter 2, page 53)
- ANSEL Character Codes and ANSEL / Unicode conversion (see Appendix B, page 130 and Appendix C, page 132)

Two Levels

GEDCOM is a two-level specification. The base level, the *GEDCOM data format* is not unlike XML; it defines the basic format and syntax. The second level, a *GEDCOM form*, is not unlike an XML Document Type Definition (DTD); it defines a particular schema on top of the basic rules. Just like in XML, the base level is a bit more than just format and syntax, but also includes items that must be present c.q. supported in every file; this is the Basic GEDCOM Language.

Chapter 1 describes the base level, the ***GEDCOM data format***, both the basic syntax and the Basic GEDCOM Language. This is a general-purpose data representation language for representing any kind of structured information in a sequential medium. Chapter 1a discusses the GEDCOM syntax and identification of structured information in general. It does not deal with the semantic content of any particular kind of data.
Chapter 1b discusses the Basic GEDCOM Language, such as the mandatory header and trailer.
Chapter 2 describes the higher level by way of one particular ***GEDCOM form***, known as the *Lineage-Linked Form*. This is the GEDCOM use by genealogical software systems to exchange data with each other.

The separation between the GEDCOM data format and GEDCOM forms is half-hearted.
The GEDCOM specification does not properly separate the GEDCOM data format and the lineage-linked from another. The GEDCOM header (in which you specify the GEDCOM form) is actually defined as part of the lineage-linked form instead of the GEDCOM data format, where it logically belongs. There is also just one GEDCOM version number, while there really should be two, one for the GEDCOM data format, and one for the lineage-linked form.

Chapter 1 Basic GEDCOM Language

Introduction

This chapter describes the core GEDCOM data representation language.
The generic data representation language defined in this chapter may be used to represent any form of structured information, not just genealogical data, using a sequential stream of characters.

Concepts

A GEDCOM file is a text file that represents a database in the form of a sequential stream of related records. A record is represented as a sequence of tagged, variable-length lines, arranged in a hierarchy. A GEDCOM line always contains a hierarchical level number, a tag, and an optional value. A line may also contain a cross-reference identifier or a pointer.
Each line, including the last one, is terminated by either a carriage return (CR), a line feed character (LF), or the carriage return/line feed combination (CR/LF). GEDCOM allows different line terminators, but each GEDCOM file must use a single line terminator throughout the file. GEDCOM 5.5.1 and earlier allow the line feed/carriage return (LF/CR) combination as a terminator, GEDCOM 5.5.5 does not.

Record types are identified by tags. Each tag identifies a GEDCOM record type. Different subrecord types may use the same tag.
Top-level records are identified by their tag. Subrecords are identified by their tag, but only fully identified by their complete tag hierarchy; all the tags of the enclosing records and the tag of the subrecord itself.

Recording records as tags with values allows record types to occur one, multiple times or zero times. It also allows the introduction of new records and subrecords without introducing incompatibility, as a reading system will continue to process the records it does understand, while ignore records it does not understand. This allows so-called user extensions.

GEDCOM files consists of records that may have subrecords. Subrecords may have subrecords again. A record without subrecords takes up exactly one GEDCOM line. A record with subrecords takes up multiple GEDCOM lines.

Each line is a GEDCOM record. Most one-line records are subrecords of an enclosing record. The enclosing records of a subrecord are known as the context of that record. The meaning of a subrecord depends on it context. A record and the subrecord it encloses are simply known as a record.

The hierarchy of records and subrecords is indicated by the level numbers. Top-level record have level number 0 (zero). Subrecords have a higher level numbers. Immediate subrecords have a level number exactly one higher than the enclosing record. It is a fatal error to skip a level.

A GEDCOM file consists of a sequence of top-level records. Each top-level record takes up one or more lines. Most top-level records have subrecords, so most top-level records consist of multiple lines. The beginning of one top-level record marks end of the previous top-level record. each top-level record continues to the next record with level number 0 or the *end of file* (EOF).

In addition to hierarchical relationships, GEDCOM defines the inter-record relationships that allow a record to be logically related to other records, without introducing redundancy. These relationships are represented by two additional, but optional, parts of a line: a cross-reference pointer and a cross-reference identifier. The cross-reference pointer "points at" a related record, which is identified by a required, matching unique cross-reference identifier. The cross-reference identifier is analogous to a primary key in relational database terminology.

Grammar

This chapter defines the grammar for the GEDCOM format. The grammar is a set of rules that specify the character sequences that are valid for creating the GEDCOM line. The character sequences are described in terms of various combinations of elements (variables and/or

constants). Elements may be described in terms of a set of other elements, some of which are selected from a set of alternative elements. Each element in the definition is separated by a plus sign (+) signifying that the items on either side of the plus sign are to be concatenated. When there is a choice of different elements that can be used, the set of alternatives are listed between opening and closing square brackets ([]), with each choice separated by a vertical bar([alternative_1 | alternative_2]). When there is only one alternative shown then the choice is optional, that is, it is the same as [alternative_1 | <NULL>]. The user can read the grammar components of the selected element by substituting any sub-elements until all sub-elements have been resolved.

A GEDCOM file consists of a sequence of top-level records, which may contain subrecords. Each record consists of a sequence of **gedcom_lines**, all contained in a sequential file or stream of characters. The simplest GEDCOM records consist of just one **gedcom_line**. The following rules pertain to the **gedcom_line**:

Grammar Rules

- **!** Long values can be broken into shorter GEDCOM lines by using a subordinate CONC or CONT tag. The CONC tag assumes that the accompanying subordinate value is concatenated to the previous line value without adding a newline. If a concatenated line value is broken at a space, then the space must be carried over to the CONC line value. The CONT record assumes that the subordinate line value is concatenated to the previous line, after inserting a newline.

  A GEDCOM writer should consistently use the same newline throughout the GEDCOM file, and that newline should be appropriate for the receiving system (`HEAD.DEST`).
  A GEDCOM reader should insert the newline appropriate for the platform it is running on.

- **!** The beginning of a new top-level record is designated by a line whose **level** number is 0 (zero).

- **!** Level numbers must be between 0 to 99 and must not contain leading zeroes, for example, level one must be 1, not 01.

- **!** Each new level number must be no higher than the previous line plus 1.

- **!** All GEDCOM records must have a value, i.e. each GEDCOM record must have a line value, subrecords or both. So, every GEDCOM record without subrecords must have a line value. The two GEDCOM records that are the exceptions to this rule are the CONT and TRLR record; empty CONT records are needed to record a sequence of empty lines, and the mandatory TRLR does not have a line value or subrecords.
  By the way, the CONC record *isn't* an exception to this rule; each CONC record must have a line value.

- **!** Any length constraints are given in code units, not bytes or characters. When wide characters are used, byte buffer sizes should be adjusted accordingly.

  The length of a text is measured in characters, code points or code units, the size of a buffer is measured in bytes.

  The *characters, code units, and code points* annotation provides a brief explanation.

- **!** The cross-reference ID has a maximum length of 22 code units, including the enclosing ‘at ’ signs (@), and it must be unique within the GEDCOM file.

GEDCOM Identifier Length

Here, the GEDCOM grammar states that GEDCOM identifiers are limited to 22 code units (characters), including the enclosing at signs. Further on, the lineage-

linked form states that identifiers are at most 22 characters, but that's 22 *without* the enclosing at signs. Thus, the GEDCOM grammar and the lineage-linked form contradict each other. The GEDCOM grammar takes precedence over any GEDCOM form.

- The maximum length of a GEDCOM identifier is 22 code units.
- The maximum length of the identifying part within the at signs is 20 code units.

References

GEDCOM Identifiers: Length

! A GEDCOM file must have *referential integrity*; The existence of a pointer does not merely imply the existence of a corresponding record, it *demands* the existence of corresponding record. A GEDCOM reader *must* make sure all pointers are valid, and must reject any GEDCOM file containing invalid pointers as corrupt.
Pointers must be to existing cross-reference identifiers for records in the same GEDCOM file, and those records must be of the right record type. Pointers to non-existent cross-reference identifiers are a fatal error. Pointers to existing cross-reference identifiers, but of the wrong record type, are fatal errors too.
Orphaned top-level records, i.e. top-level record that aren't pointed to, are perfectly legal. A GEDCOM reader must import orphaned records just like other records. A smart GEDCOM reader may issue a warning about orphaned top-level records, but should never do so for the special case of a GEDCOM file that contains just one record.

! The length of a GEDCOM tag is a maximum of 31 code units.
All characters of the tag are significant.

! The total length of a GEDCOM line, including level number, cross-reference number, tag, value, delimiters, and terminator, *must not* exceed 255 code units.
An ostensible GEDCOM 5.5.5 file containing longer lines isn't a GEDCOM 5.5.5 file. A GEDCOM 5.5.5 reader *must not* accept longer lines, but *must* reject the file as invalid, with an error message such as “Line is too long. This is not a GEDCOM 5.5.5 file.”.

! Empty lines and leading white space are illegal. Systems generating GEDCOM 5.5.5 files must not create empty lines or start any lines with leading white space. Systems reading GEDCOM 5.5.5 files must not accept empty lines or lines starting leading white space.
A GEDCOM 5.5.5 writer *does not* create any empty lines or leading white; if it does, it isn't a GEDCOM 5.5.5 writer.
A GEDCOM 5.5.5 reader *must not* tolerate empty lines or leading white space, but *must* reject such files as not GEDCOM.

Grammar Syntax

A **gedcom_line** has the following syntax:

gedcom_line:=

level + [delim + xref_ID] + delim + tag + [delim + line_value] + terminator

for example:
1 NAME Will /Rogers/

The components used in the pattern above are defined below in alphabetical order. Some of the components are defined in terms of other primitive patterns. The spaces used in the patterns below are only to set them apart and are not a part of the resulting pattern. Character constants are specified by their Unicode code point, using the notation introduced with Unicode 2.0; a Unicode code point is represented as U+*nnnn*, where *nnnn* is the hexadecimal value, e.g. U+0020 is the Space character. Character constants that are separated by a (-) dash represent any character within that range from the first constant shown to and including the second constant shown.

alpha:=

[U+0041 - U+005A | U+0061 - U+007A]

where:

U+0041 - U+005A = A through Z
U+0061 - U+007A= a through z

alphanum:=

[alpha | digit]

delim:=

space

where:

space = U+0020, the Space character

The explicit inclusion of the delim definition as the space character is no mistake. It stresses that the delimiter is a space character and nothing else.

digit:=

U+0030 - U+0039
where:
U+0030 - U+0039 = One of the digits 0, 1,2,3,4,5,6,7,8,9

escape:=

U+0040 + U+0023 + escape_text + U+0040

where:

U+0040 = @
U+0023 = #

A GEDCOM escape sequence begins and ends with an at sign.
An escape sequence must not contain any at sign and must not be followed by an at sign; an escape sequence must be followed by either a delim (the space character) or a terminator.

escape_text:=

[alphanum | escape_text + alphanum | escape_text + space]

where:

space = U+0020, the Space character

The escape_text is the part of an escape between the opening and closing at sign (@). Escape sequences are defined by a GEDCOM form for that GEDCOM form.

GEDCOM 5.5.1 allows almost any Unicode character inside escape sequences. GEDCOM 5.5.5 restricts escape sequences to alphanumerical characters and the space character. Notice that escape_text must start with an alphanumerical character, that it must not start with a space.

The space character is allowed but deprecated; it is only included to keep the already existing escape sequence `@#DFRENCH R@` legal.

identifier_string:=

[alphanum | alphanum + identifier_string]

level:=

[digit | non_zero_digit + digit]

The level number consists of one or two digits. It may be zero, but must not start with leading zeroes.

line_char:=

all legal characters with some exceptions and one special case

specifically disallowed:
U+0000 - U+001F, except U+0009 = most C0 control characters
U+00FF = Delete character

specifically allowed:
U+0009 = Horizontal Tab

special case:
U+0040 + U+0040 = @@

Most control characters in the range U+0000 - U+001F (C0 Control Set) are disallowed. The one exception is U+0009, the Horizontal Tab. Text may contain horizontal tabs.
Note that Carriage Return (U+000D) and Line Feed (U+000A) must not occur inside text. Line breaks are supported through the `CONT` record. The use of the space character ( ), number-sign (#) and underscore (_), is perfectly legal in text.

Legal Characters

All text must abide by the rules of the character set used. The character set used determines which code points are legal. The character set used determines which code point combinations are legal. Use string functions to make sure that any line breaks are between characters, not inside characters.

Special Case: At sign (@)

The At sign (@) is legal in text, but because of its special function in GEDCOM, there's a twist. Text is not allowed to contain a single at sign (@), but must always include two consecutive at signs (@@) instead, to clearly distinguish an at sign that is just text from an at sign starting a cross-reference or escape sequence.
This is similar to backward slashes (\) having to be doubled inside C and C++ text strings.

This at sign rule affects all textual line values, but particularly the EMAIL line value, <ADDRESS_EMAIL>, introduced with GEDCOM 5.5.1, as email addresses always contain a single at sign. Within a GEDCOM file, that single at sign must be represented by a double at sign.

A GEDCOM writer must export a double at sign instead of single at sign, and a GEDCOM reader must import each double at sign as a single one. Exporting a single at sign where a double at sign should be exported is a serious GEDCOM grammar mistake. A GEDCOM 5.5.5 reader *must not* interpret a single at sign as if it were a double at sign, but must report a fatal error and exit.

dates

GEDCOM writers should pay special attention to at signs in date fields. When an application

does not provide support for a particular calendar, a user may choose to enter a date complete with the <DATE_CALENDAR_ESCAPE> for the calendar, e.g. "@#JULIAN@ 12 Oct 1492" (without the quotes). The at signs within such dates should *not* be doubled.

Not Deprecated

This feature complicates GEDCOM handling, so it is desirable to mark this GEDCOM feature as deprecated, and eventually get rid of it, but as long as some records can contain GEDCOM pointers as well as free text, this feature allows GEDCOM readers to distinguish between an at sign that starts a pointer (single at sign), and one that is merely an at sign in some text (double at sign).

line_item:=

[escape | line_text | escape + delim + line_text]

This line_item definition is different from the GEDCOM 5.5.1 definition. It explicitly restricts line_item to just three different formats.

The explicit inclusion of nothing but an escape sequence as a possible line value is as intended. It has been used in GEDCOM 5.0, it is deprecated now.

line_text:=

[line_char | line_text + line_char]

line_text is text that neither is nor contains a pointer or an escape sequence.

line_value:=

[pointer | line_item]

non_zero_digit:=

U+0031 - U+0039
where:
U+0031 - U+0039 = One of the digits 1,2,3,4,5,6,7,8,9

null:=

nothing

pointer:=

xref_ID

tag:=

[[U+005F] + alphanum | tag + alphanum]

where:

U+005F = _ (underscore)

GEDCOM tags are Case-Sensitive.
Each tag has a single correct casing.
All Basic GEDCOM Language tags are UPPERCASE.

- A GEDCOM writer must use the correct casing for tags.
- A GEDCOM reader must recognise tags through case-sensitive comparison.
- A GEDCOM reader must not recognise tags through case-insensitive comparison.

GEDCOM tags are generally restricted to alphanumeric characters. Tags need not start with

a letter, but may start with a digit. Tags may additionally start with an underscore; this is allowed because the Lineage-Linked Form demands that non-standard tags start with an underscore.

terminator:=

[carriage_return | line_feed | carriage_return + line_feed]

where:

carriage_return = U+000D, the Carriage Return character
line_feed = U+000A, the Line Feed character

A GEDCOM writer must not use different terminators for different lines.
A GEDCOM writer must use a single terminator throughout a GEDCOM file.

A GEDCOM file is a text file, which you can load into an text editor. Like any other text file, a GEDCOM file must use a single line terminator throughout the file, and the occurrence of either ᶜᴿ or ᴸᶠ outside the line terminator is a fatal error: not a GEDCOM file, not even a text file.

A GEDCOM writer should not pick the terminator appropriate for the platform it runs on, but the terminator appropriate for the destination system (`HEAD.DEST`). GEDCOM writers should use ᶜᴿ/ᴸᶠ when unsure which terminator to pick. Web applications should always use ᶜᴿ/ᴸᶠ.

Starting with GEDCOM 5.5.5, the terminator must be either ᶜᴿ, ᴸᶠ or ᶜᴿ/ᴸᶠ and may no longer be ᴸᶠ/ᶜᴿ.
This is correction of a mistake; ᴸᶠ/ᶜᴿ should never have been included as an option; its inclusion was a problem instead of a solution. Text editors and GEDCOM readers expect ᶜᴿ, ᴸᶠ and ᶜᴿ/ᴸᶠ, and are likely to get seriously confused when presented with ᴸᶠ/ᶜᴿ. Even though it has been legal for decades, not a single genealogy applications uses ᴸᶠ/ᶜᴿ as a GEDCOM line terminator. The ᴸᶠ/ᶜᴿ terminator only appears in test files specifically crafted to test support for it.

References

- GEDCOM Lines
- Famberry GEDCOM

xref_ID:=

at + identifier_string + at

where:

at = U+0040, the At Sign (@)

GEDCOM 5.5.5 demands that the identifier_string be alphanumerical.

Description of Grammar Components

alpha:=

The alphabetic characters, both uppercase and lowercase, i.e. A through Z and a through z.

delim:=

The **delim** (delimiter), a single space character, terminates both the variable-length **level** number and the variable-length **tag**. Note that space characters may also be present in a

value.

escape:=

The **escape** is a character sequence in the grammar used to specify special processing, such as for switching character sets or for indicating an inclusion of a non-GEDCOM data form into the GEDCOM structure.

An escape sequence should be separated from any line item following it by a delimiter (space character). The delimiter following the escape sequence is not part of the escape sequence (the GEDCOM 5.5.1 grammar erroneously states that it is). Failure to include a delimiter between an escape sequence and a line item following it is a syntax error.

escape_text:=

Legal escape_text values and their meaning are defined by each GEDCOM form for that GEDCOM form.

level:=

The **level** number works the same way as the level of indentation in an indented outline, where indented lines provide detail about the item under which they are indented. A line at any level L is enclosed by and pertains directly to the *nearest preceding line* at level L-1. The level L may increase by 1 at most. Level numbers *must not* contain leading zeroes, for example level one must be `1`, not `01`.

The enclosed subordinate lines at level L are said to be in the context of the enclosing superior line at level L-1. The interpretation of a **tag** must be in the context of the **tag**s of the enclosing line(s) rather than just the tag by itself.
Take the following record about an individual's birth and death dates, for example:

0 INDI
1 BIRT
2 DATE 12 MAY 1920
1 DEAT
2 DATE 1960

In this example, the expression DATE 12 MAY 1920 is interpreted within the INDI (individual) BIRT (birth) context, representing the individual's birth date. The second DATE is in the INDI.DEAT (individual's death) context. The complete meaning of DATE depends on the context.

GEDCOM writers *must not* insert empty lines to highlight the end of one and beginning of another record nor indent lines based on their level number to increase readability. The GEDCOM grammar is crystal clear that GEDCOM files do not contain empty lines and GEDCOM lines do not contain leading white space.
GEDCOM 5.5.5 readers must reject any such files as *not a GEDCOM file*.

Anyone viewing or editing a GEDCOM file can enjoy increased readability though colour coding, indentation and other conveniences by using a dedicated GEDCOM editor or text editor with GEDCOM support.

line_char:=

Characters legal within a textual line value.

line_text:=

A purely textual line value (no pointers or escape sequences).

line_value:=

The **line_value** identifies an object within the domain of possible values allowed in the context of the **tag**. The combination of the tag, the **line_value**, and the hierarchical context of the supporting **gedcom_line**s provides the understanding of the enclosed **value**s. This

domain is defined by a given GEDCOM form. The lineage-linked form defined in Chapter 2 Lineage Linked-Form, page 53 is one such GEDCOM form.

Values whose source information contains illegible parts of the value should be indicated by replacing the illegible part with an ellipsis (...).

Values are generally not encoded in binary or other abbreviation schemes for reducing space requirements, and they are generally constrained to be understandable by a typical user without decoding. This is intended to reduce the decoding burden on the receiving software.

GEDCOM files are text files with a fair amount of repetition, and as such compress well using common file compression methods.

Line Value Ellipsis

The instructions to use ellipsis for illegible parts is problematic, because original text may contain ellipsis already.

FamilySearch probably meant this instruction for names in particular; use ellipsis for illegible parts of a name.

The **line_value** within the context of a tag hierarchy of **gedcom_line**s represents one piece of information and corresponds to one field in traditional database or file terminology.

pointer:=

A **pointer** stands in the place of the record or context identified by the matching **xref_ID**. The use of **pointer**s is explicitly defined within the GEDCOM form, such as the Lineage-Linked GEDCOM Form defined in Chapter 2 (p. 53).

The **pointer** is a link to a top-level records with an identifier inside the same GEDCOM file. The GEDCOM grammar does not support linking to records in other files or to subrecords.

A pointer must be both legal and valid. A pointer is legal if it conforms to the pointer syntax. A pointer is valid if it matches the identifier of a record and that record is of the right type.

The pointer and matching record need not appear in any particular order; both *backward references* (record appeared already) and *forward references* (pointer appears first) are legal. GEDCOM readers typically create a look-up table of record identifiers while reading a GEDCOM file, and verify the validity of pointers against that table after reading the entire file.

The GEDCOM 5.5.1 specification mentioned two possible future features, for which it already complicated the parsing of pointers. The two obsolete future features are linking to records in other files, and linking to subrecords, which demand that GEDCOM readers give special treatment to colons and exclamation marks respectively.
The GEDCOM 5.5.5 specification does not allow semicolons or exclamation marks in pointers.

Verifying whether pointers are legal was complex already, as GEDCOM 5.5.1 allowed many different characters inside pointers. GEDCOM 5.5.5 simplifies pointers by restricting pointers to alphanumeric characters.

tag:=

A **tag** consists of a variable length sequence of **alphanum** characters. All user-defined tags that have not been defined in the GEDCOM standard, must begin with an underscore character (U+005F.

A tag defines the meaning of the **line_value** within the context of the enclosing tags, and

contributes to the meaning of the enclosed subordinate lines. Specific **tag**s are defined in Appendix A, page 121.
The presence of a tag together with a value represents an assertion which the submitter wishes to communicate to a receiver. A tag without a value does not represent an assertion. If an optional record (the entire line; level, *tag*, line value and terminator) is absent, no assertion is made. Information of a negative nature (such as knowing positively an event did not occur) is handled through the semantic definition of a different tag and its accompanying value to assert that information explicitly.

Although all the tags of the lineage-linked form are only three, four or five characters long, systems must be able to handle all legal tags.
The GEDCOM 5.5.1 specification demanded that tags be unique in the first 15 characters (code units), and thus allowed for wild variation beyond the first 15 characters. The GEDCOM 5.5.5 specification simplifies GEDCOM parsing by demanding that all tags are unique, period. For maximum backward compatibility, developers are still advised to avoid tags that aren't unique in the first 15 characters.

How different record types and their tags may be combined is defined by a GEDCOM form. A GEDCOM form defines the valid record and subrecords, their line values and possible cross-references.

Valid **tag**, **line_value**, **xref_ID**, and **pointer** combinations are constrained by the GEDCOM form defined for representing a given kind of information. Chapter 2 Lineage-Linked Form, page 53 defines the commonly used GEDCOM form.

Tags and records aren't the same thing. The FamilySearch GEDCOM specifications sometimes uses “tag” to mean “record”. That is confusing. A conscious effort has been made to always use the right term throughout the GEDCOM 5.5.5 specification.

terminator:=

The **terminator** delimits the variable-length **line_value** and signals the end of the **gedcom_line**.
The valid terminators are Carriage Return (CR), Line Feed (LF) and the CR/LF combination. The LF/CR combination is no longer allowed.

GEDCOM writers must pick one line terminator, and use only that line terminator throughout the entire GEDCOM file.

xref_ID:=

(See **pointer**), page 35
The **xref_ID** is formed by any arbitrary combination of alphanumeric characters. No meaning is attributed by the receiver to any part of the **xref_ID**, other than its **unique** association with the associated record.

Examples:
The following are examples of valid but unrelated GEDCOM lines:

```
0 @I1234@ INDI
. . .
1 AGE 13y
. . .
1 CHIL @I1234@
. . .
1 NOTE This is a note field that is
2 CONT continued on the next line.
```

The first line has a **level** number 0, a **xref_ID** of @I1234@, an INDI **tag**, and no **value**.
The second line has a **level** number 1, no **xref_ID**, an AGE **tag**, and a **value** of 13y (13 years).
The third line has a **level** number 1, no **xref_ID**, a CHIL **tag**, and a **value** of a **pointer** to a **xref_ID** named @I1234@.

History: FamilySearch GEDCOM 5.5.1 allowed many different characters inside pointers, even spaces, and demanded special handling of colons and exclamation marks in support of obsolete future features.
GEDCOM 5.5.5 simplifies parsing by limiting pointers to alphanumeric characters.

Implementation note: In practice all systems use table indices to generate unique identifiers; the identifier for the first record in the individual table is @I1@, the identifier for the fourth record in the sources table is @S4@, and so on.
The receiving system may ignore this numbering system, but often chooses to use the same indices as used in the source system. Users of multiple applications like this, as it allows searching for individuals and family groups using the same numbers as used in the source system.

References

- Common GEDCOM Identifier Naming Convention

White Space

Spaces & Tabs

For purposes of the GEDCOM grammar, *white space* is defined as spaces and tabs (horizontal tabs, vertical tabs are illegal). White Space includes spaces and tabs only, line terminators are not considered white space.

Unicode has many more characters in the white space category, but those characters are not relevant to GEDCOM parsing. GEDCOM readers and writers *must not* take any special action for those characters.

GEDCOM 5.5.1 versus 5.5.5

GEDCOM 5.5.1 did not allow tabs, but many GEDCOM 5.5.1 files do contain tabs anyway. Few genealogy software developers ever restricted user from entering horizontal tabs, mostly because they did not even notice the restriction.
GEDCOM 5.5.5 allows user text to contain horizontal tabs.

Previous GEDCOM versions do not explicitly allow or disallow trailing white space on GEDCOM lines. GEDCOM 5.5.1 mentions that many systems will strip trailing spaces when reading GEDCOM lines, and mentions it as fact that should be taken into account, which seems a tacit approval of that practice.
GEDCOM 5.5.5 changes that: a GEDCOM reader *must not* strip trailing white space.

Superfluous versus Significant

This specification distinguishes between superfluous and significant white space.
Superfluous white space is white space that serves no purpose, and really should not be there, such as white space before or after a place name in a place name field. Superfluous white space is often the result of an edit error, and its continued presence is a normalisation issue. Superfluous white space can and should be trimmed from application records.
Significant white space is white space that must not be lost, such as a tab or space between two words. Significant white space includes all white space entered into free-form text fields, regardless of senseless some of that white space may seem to a human.

GEDCOM Lines

GEDCOM lines *must not* start with leading space.